

IMPLEMENTATION DESIGN

信创智能客服聊天业务与智能体实现设计说明

接口契约 · Agent 编排 · RAG 检索 · 工具治理 · Redis/MongoDB 上下文存储

文档信息	内容
版本	v1.0
编制日期	2026-08-27
适用项目	C:\work_learn\XinChuang_pc
主要读者	后端、AI、前端、测试、运维及技术评审人员
依据	当前仓库代码、DDL、OpenAPI/SSE 契约及主流 Agent 官方实践

核心结论 保留 Java 业务服务作为鉴权、资源归属、幂等、请求状态与 SSE 出口；Python Agent 服务使用 LangChain + LangGraph 承载显式状态图、RAG 和受控工具。MySQL 仅保存用户可查阅的提问与最终回答；Redis 保存近期模型上下文，MongoDB 持久化 Agent 上下文、摘要、工具观察和检查点。模型上下文与界面聊天记录是两套语义不同的数据。

文档状态：可进入详细设计评审与任务拆分

目录

- 1. 文档目标与边界
- 2. 当前代码现状与差距
- 3. 目标总体架构与职责边界
- 4. 智能体编排实现
- 5. 检索增强生成（RAG）实现
- 6. 工具调用与安全治理
- 7. 上下文、记忆与数据存储
- 8. 公共聊天接口详细设计
- 9. Java 与 Python 内部接口
- 10. SSE、状态机、一致性与异常处理
- 11. 代码结构与实施步骤
- 12. 测试、评测、可观测与验收
- 附录 A. 数据结构建议
- 附录 B. Redis Key 建议
- 附录 C. 参考资料与项目依据

阅读提示 第 2 章区分“已经实现”和“仅有契约/骨架”；第 8、9 章中的接口状态以 2026-08-27 仓库代码为准。

1. 文档目标与边界

本文面向本项目聊天业务与智能体部分的后续编码，目标不是重复既有架构文档，而是把现有代码、缺失实现和可落地的目标方案连接起来。所有建议均以当前 Spring Boot、Vue、MyBatis、MySQL、Redis 代码为起点，并允许新增 Python Agent 服务、MongoDB 与 Qdrant。

1.1 交付范围

- 聊天会话、历史消息、流式问答、请求恢复、取消、引用和反馈的公共接口设计。
- Java 业务服务与 Python Agent 服务之间的内部接口、鉴权和流式转发约定。
- 智能体编排状态图、节点职责、工具调用策略、RAG 检索链路和降级策略。
- 近期模型上下文在 Redis、持久 Agent 上下文与运行状态在 MongoDB 的存储模型，以及它们与 MySQL 用户可见聊天记录的严格边界。
- 状态机、事务边界、幂等、错误码、安全、可观测、测试与迭代计划。

1.2 不在本次范围

- 不直接实现模型训练、Embedding 模型训练或 GPU 推理平台。
- 不一次性重构认证、管理端、工单和知识库全部业务；仅描述其与聊天/工具的交界面。
- 不允许智能体直接执行任意 SQL、任意 URL 请求、文件系统命令或未经确认的高风险写操作。

1.3 设计原则

原则	落地方式
代码编排优先	固定主流程由状态图控制；LLM 只负责分类、规划和生成等需要推理的节点。
单一对话负责人	一个客服 Supervisor 负责最终答复；专家能力默认作为工具/子图，不轻易切换用户侧身份。
最小权限	工具白名单、参数 Schema、服务端注入用户身份、超时/次数/结果大小上限。
有依据再回答	知识性结论必须通过证据门控和引用校验；证据不足时澄清、拒答或引导工单。
可恢复	请求状态、Agent run、检查点和最终结果均可按 requestId 恢复；SSE 断线不等于业务失败。
上下文分离	MySQL 聊天记录不进入模型上下文；Redis/MongoDB 保存经过筛选、结构化和压缩的 Agent 上下文。

2. 当前代码现状与差距

本章结论来自对当前仓库 Controller、Service、DTO/VO、Mapper XML、DDL、OpenAPI、SSE Schema 和 Vue 聊天客户端的核对。

2.1 已有技术基线

层次	当前实现
后端	Java 21、Spring Boot 4.1.1、Spring MVC、Validation、MyBatis 4.1.0、MySQL、Spring Data Redis、JWT。
前端	Vue 3 + TypeScript + Pinia；已实现 POST SSE 的 fetch 流读取、事件序号校验、终态校验、取消和 Mock 流。
聊天数据	MySQL 已定义 chat_session、chat_request、chat_message、message_citation、message_feedback、api_idempotency_record。
契约	openapi.yaml 已定义 8 个公共聊天接口；sse-events.schema.json 已定义 8 类 SSE 事件。
AI 骨架	InternalDTOs/InternalVOs 已定义聊天、索引、评测和 usage 数据结构；

层次	当前实现
	InternalAiController/Service 仍为空，LangChain/LangGraph 服务尚未落地。
缺失组件	仓库内尚无 Python Agent 服务、MongoDB/Qdrant 客户端与实际模型调用实现。

2.2 接口完成度

接口	代码状态	关键说明
GET /sessions	已实现	按 JWT userId 隔离、分页、关键字、真实 COUNT。
PATCH /sessions/{id}	已实现	按主键+userId 更新；404 隐藏资源归属。
DELETE /sessions/{id}	已实现	写 deleted_at 的软删除。
GET /sessions/{id}/messages	部分实现	消息可分页；citations 当前恒为空，且 SQL 尚未限制为用户可见的最终消息。
POST /chat/stream	待实现	DTO、VO、OpenAPI、前端消费已存在，后端 Controller/Service 缺失。
GET /chat/requests/{id}	待实现	VO 和表已存在，ChatRequestMapper 为空。
POST /chat/requests/{id}/cancel	待实现	前端会调用；需要 Redis 取消标记和状态机。
PUT /messages/{id}/feedback	待实现	DTO/表/前端已存在；Mapper/Service/Controller 缺失。
/internal/ai/v1/**	仅数据结构	Controller/Service 为空，尚无 Python 服务。

2.3 必须在编码前修正的契约差异

- 反馈枚举以当前代码与 OpenAPI 的 up/down/null 为准；旧文字文档中的 HELPFUL/UNHELPFUL 不再采用。
- 前端为重命名、删除、取消、反馈发送 Idempotency-Key，但当前 Controller 未声明或处理该请求头；实现时必须统一。
- 历史消息 Mapper 目前未批量加载 message_citation，导致前端无法展示真实引用。
- 当前 SSE meta 要求 assistantMessageId，旧实现思路会先插入 STREAMING 占位消息；按本项目新边界应取消占位消息，meta 中 assistantMessageId 改为可空/移除，最终 ID 只在 done 返回。
- 历史分页按消息 id 正序且使用 offset。展示历史可以保留；Agent 组装上下文必须新增“倒序取最近 N 条再正序还原”的独立查询，不能复用第一页历史接口。
- application.yml 中存在仅适用于本地开发的默认密码和 JWT secret；生产环境必须取消弱默认值并由密钥系统注入。
- 现有架构文档写过 Spring Boot 3.5.x，但 pom.xml 当前实际为 4.1.1；依赖与实现说明以 pom.xml 为准。

现状判断 聊天 UI 和数据契约已经超过后端实现进度。最短路径不是重写前端，而是先补齐 Java Chat 写链路与请求状态，再接入 Python Agent。

3. 目标总体架构与职责边界

3.1 推荐部署单元

组件	职责	禁止承担
Vue Web	输入、SSE 展示、取消、恢复、引用与反馈交互	模型路由、知识库白名单、userId 决策
Java Business API	JWT、权限、幂等、会话/请求状态、SSE 网关、业务工具、审计	开放式 Agent 循环、向量检索实现
Python Agent Service	LangGraph 状态图/检查点；LangChain 模型、Prompt、Retriever、Tool；上下文、RAG 与编排	最终用户权限、任意业务写入
Redis	近期上下文、取消标记、并发配额、短锁、SSE 短期重放	不可恢复的唯一真相
MongoDB	持久 Agent 上下文、压缩摘要、runs/checkpoints、检索/工具观察、可选长期记忆	用户界面聊天记录、最终业务权限
MySQL	用户可见提问/最终回答、会话归属、chat_request 状态、幂等、反馈、引用、工单与知识元数据	模型上下文组装与 Agent 中间状态
Qdrant	知识切片的 dense/sparse 向量与过滤检索	文档原件、发布状态唯一真相

3.2 主链路

步骤	处理
1	浏览器向 Java 提交 POST /api/v1/chat/stream；Java 验证 JWT、参数、容量和 Idempotency-Key。
2	Java 在 MySQL 短事务中创建/校验会话、chat_request 和用户提问记录，随后发送 meta；此时不创建助手占位消息。
3	Java 以 HMAC 调用 Python 内部 SSE；Python 从 Redis 读取热上下文，缺失时仅从 MongoDB 的 Agent 上下文回源，不读取 MySQL 聊天记录。
4	Python 执行输入安全、意图路由、FAQ 快路径或 RAG/工具子图，并把 status/delta/citation/usage 流回 Java。
5	完成后 Python 将本轮经筛选/压缩的 Agent 上下文、摘要、工具观察和 run 持久化到 MongoDB，并刷新 Redis 热上下文。
6	Java 只把用户提问与最终展示答案写入 MySQL，并更新请求终态/引用；上下文持久化与展示记录均成功后才发送 done。

3.3 为什么采用显式状态图

客服问答的核心路径、超时、权限、引用与终态具有强约束，不适合让模型无限循环决定所有步骤。本项目明确使用 LangGraph StateGraph 固化主路径、条件分支、检查点和恢复；使用 LangChain 提供模型适配、Prompt 模板、Retriever、Tool、结构化输出与回调能力。只在“意图分类、查询改写、工具选择、回答生成”等节点使用结构化模型输出。

多智能体策略 首期不建设多个可自由对话的自治 Agent。一个 Supervisor 负责用户侧输出；诊断、知识检索、工单建议等能力以工具或受限子图存在。只有当提示词、工具集和评测集明显不同且收益可验证时，才拆分为专家 Agent。

4. 智能体编排实现

4.1 AgentState 建议

```
class AgentState(TypedDict):
    request_id: int
    session_id: int
    user_id: int                # 仅内部可信字段
    user_message: str          # 已完成必要脱敏
    history: list[Message]
    conversation_summary: str | None
    policy: AgentPolicy
    safety: SafetyDecision | None
    intent: IntentDecision | None
    rewritten_queries: list[str]
    retrieved_chunks: list[Chunk]
    tool_calls: list[ToolCallRecord]
    answer_draft: str | None
    citations: list[Citation]
    usage: UsageAggregate
    retry_count: dict[str, int]
    final_status: str | None
    error: AgentError | None
```

状态中只保存完成流程所需的数据。原始 JWT、Cookie、数据库密码、模型密钥和未脱敏高敏文本不得进入图状态或 Trace。requestId 是跨 Java、Python、MongoDB、Redis、Qdrant、模型调用和日志的统一关联键。

4.2 推荐状态图

序号	节点	职责	下一步
01	initialize	校验内部签名、加载 policy、创建 run/checkpoint；发 queued。	input_guard
02	input_guard	敏感信息识别、注入检测、输入长度/领域检查。	拒绝或 context_load
03	context_load	Redis 热上下文命中；否则仅从 MongoDB Agent context 回源并重建摘要窗口。	intent_router
04	intent_router	结构化输出 intent、confidence、need_retrieval、need_tool、clarify。	FAQ/RAG/tool/clarify

序号	节点	职责	下一步
05	faq_fast_path	规范化问题，查 Redis FAQ 缓存和 MySQL 已发布 FAQ。	证据足够则 answer
06	retrieve	查询改写、dense+sparse 召回、权限/版本过滤、RRF、重排。	evidence_gate
07	evidence_gate	判断覆盖度、冲突和最低相关度；不够则澄清/工单。	tool_plan/answer
08	tool_plan	从服务端白名单选择 0~N 个工具并生成严格参数。	tool_guard
09	tool_guard/execute	鉴权、风险级别、确认、超时、幂等；最多 3 次。	observe/answer
10	answer	基于上下文、证据和工具结果生成；不得编造来源。	output_guard
11	output_guard	引用一致性、敏感信息、危险操作、格式和完整性校验。	修复 1 次或 finalize
12	finalize	持久化 MongoDB Agent context/run/usage，更新 Redis；通知 Java 写 MySQL 最终展示答案。	END

4.3 路由策略

意图	路由	说明
GREETING/SMALL_TALK	直接生成	不检索、不调用业务工具，限制输出长度。
FAQ_EXACT	FAQ 快路径	缓存或 MySQL 精确/高置信匹配，直接带来源回答。
KNOWLEDGE_QA	RAG	手册、兼容性、故障排查、政策类问题。
DIAGNOSIS	RAG + 诊断子图	按设备型号/系统/现象补槽位，生成可执行排查步骤。
TICKET	工单工具	先生成草稿；提交写操作前需要用户明确确认。
OUT_OF_SCOPE/UNSAFE	拒答/转人工	说明边界，不暴露内部规则。

4.4 循环、重试与终止

- 同一 run 的模型回合默认不超过 4，工具调用总数不超过 3，单工具超时 3 秒、工具总预算 10 秒。
- 只有无副作用节点允许自动重试；模型 429/5xx 最多指数退避 2 次，检索最多 1 次重试。
- 写工具不自动重试，除非携带稳定 Idempotency-Key 且服务端声明幂等。
- 出现用户取消、总超时、证据不足、安全拒绝、工具预算耗尽或终态写入失败时必须结束图，不允许开放式自循环。

4.5 流式输出映射

图阶段	SSE stage/event	用户可见提示
排队/初始化	status: queued	请求已受理，正在准备
输入安全	status: safety	正在检查问题
意图路由	status: intent	正在理解问题
FAQ/RAG	status: retrieval	正在检索知识库
生成	status: generation + delta*	正在组织答案
输出校验	status: validation	正在核对引用和内容
成功	citation? + usage? + done	终态，连接关闭
失败	error	终态，包含可安全展示的错误

5. 检索增强生成（RAG）实现

5.1 索引管线

- 1. Java 接收/管理文档，完成文件类型、大小、病毒扫描、SHA-256、版本和发布状态记录；对象原文进入 S3/MinIO。
- 2. 可靠索引任务通过 /internal/ai/v1/index-jobs 交给 Python；重复 jobId + attemptNo 必须幂等。
- 3. Python 解析 PDF/Office/HTML，保留标题层级、页码、表格和来源定位；按语义段落切片，建议 400~800 tokens、重叠 60~120 tokens。
- 4. 为每个 chunk 生成 dense 向量和 sparse/BM25 表示，写入 Qdrant；payload 包含 knowledgeBaseId、documentId、versionId、chunkIndex、title、page、status、aclTags、contentHash。
- 5. 索引成功后回调 Java；只有 Java 确认已发布版本后，检索才允许使用该 chunk。

5.2 在线检索流程

阶段	实现要求
问题规范化	统一空白、大小写、型号别名和操作系统别名；保留序列号、错误码等精确词。
上下文改写	结合最近对话把省略指代改写为独立查询；生成 1 个主查询，最多 2 个补充查询。
并行召回	dense 语义 Top 30 + sparse/关键词 Top 30；精确错误码/型号可额外查 MySQL FAQ。
合并	用 RRF 合并不同量纲的排名；没有离线标注集时避免直接线性相加原始分数。

阶段	实现要求
强制过滤	knowledgeBaselId、tenant/user ACL、published version、语言、产品线、有效期。
重排	对合并后 Top 20 使用 cross-encoder/LLM reranker，保留 Top 5~8。
证据门控	检查最高分、覆盖度、来源冲突和问题所需字段；不足则追问或转工单。
引用快照	生成前固定 chunkId/title/page/snippet/contentHash，回答完成后写引用快照，避免来源更新后无法追溯。

5.3 生成约束

- Prompt 明确区分 system instructions、用户内容、检索内容和工具结果；检索文档中的指令一律视为不可信数据。
- 产品事实、维修步骤、兼容性和政策类陈述必须能映射到 citation；通用寒暄无需引用。
- 模型先生成结构化 answer + citation_ids，再由服务端把 citation_ids 转成最终 CitationVO，禁止模型自行拼 URL。
- 引用校验失败可做一次受控修复；仍失败则删除无依据断言或返回证据不足。

5.4 降级顺序

故障	降级
Qdrant 不可用	尝试已发布 FAQ 精确/关键词检索；否则明确说明知识检索暂不可用。
重排模型失败	使用 RRF 排名结果，但提高证据门槛并减少断言。
主模型 429/5xx	按 modelRoute 切换备用模型；首个 delta 后不得自动整段重放。
MongoDB 暂不可用	Redis 命中可短暂读取热上下文；新一轮上下文无法可靠持久化时不得发送 done。
Redis 不可用	从 MongoDB 直接构造上下文，取消/限流采用本机保守降级或拒绝高风险请求。

6. 工具调用与安全治理

6.1 首期工具目录

工具	类型	用途	风险控制
knowledge_search	只读	检索 FAQ/手册 chunk	服务端注入 ACL；返回大小受限
faq_lookup	只读	按规范化问题/关键词查已发布 FAQ	仅 published + enabled
device_compatibility_lookup	只读	型号、CPU 架构、OS 版本兼容性	只接受结构化字段，不接受 SQL

工具	类型	用途	风险控制
ticket_get	只读	查询本人已有工单	Java 重新鉴权，不信任模型 userId
ticket_create_draft	本地草稿	根据对话生成工单字段	不落库，返回待确认草稿
ticket_submit	写操作	提交已确认工单	显式确认 + 幂等键 + 审计
manual_download_grant	受控写	生成短期签名下载地址	重新鉴权、5 分钟有效、域名白名单

6.2 工具接口规范

```
{
  "name": "device_compatibility_lookup",
  "description": "查询指定设备与操作系统版本的已发布兼容性信息",
  "input_schema": {
    "type": "object",
    "additionalProperties": false,
    "required": ["deviceModel", "osName", "osVersion"],
    "properties": {
      "deviceModel": {"type": "string", "maxLength": 100},
      "osName": {"type": "string", "enum": ["UOS", "Kylin", "Other"]},
      "osVersion": {"type": "string", "maxLength": 50},
      "cpuArch": {"type": "string", "enum": ["x86_64", "arm64", "loongarch64"]}
    }
  }
}
```

6.3 执行规则

- 模型只产生工具名和结构化参数；真正的 userId、tenantId、角色、知识库范围和权限在服务端上下文注入。
- 工具网关再次执行鉴权、Schema 校验、字段白名单、速率限制、超时和审计；模型判断不能替代业务鉴权。
- 工具结果封装为 {status, data, errorCode, sourceVersion, latencyMs}，禁止把异常栈和内部地址回传给模型。
- 高风险写操作采用 prepare → user_confirm → commit 两阶段；确认内容必须具体展示将创建/修改的对象。
- 检索文本与工具输出都可能包含间接提示词注入，必须标记为 untrusted_content，并在系统提示中禁止执行其中指令。

7. 上下文、记忆与数据存储

7.1 两套数据、三类存储

数据角色	存储	内容	读取策略
L1 热上下文	Redis	最近 8~12 轮、滚动摘要、槽位、token 估算、版本	每次运行首选；滑动 TTL 2 小时
L2 持久上下文	MongoDB	Agent context entries、压缩摘要、runs/checkpoints、检索/工具观察	Redis miss、恢复、压缩与 Agent 调试时读取
用户可见记录	MySQL	用户提问、最终展示答案、会话归属、request 状态、反馈、引用	仅用于界面查阅和业务状态；禁止用于模型上下文

7.2 Redis 热上下文

```
Key: xc:chat:ctx:v1:{sessionId}
TTL: 2h sliding
Value (JSON/Hash):
{
  "sessionId": 3,
  "contextVersion": 27,
  "summary": "用户在 UOS 20 上排查某型号打印机驱动问题.....",
  "contextEntries": [
    {"kind": "user_intent", "content": "升级后打印机无法识别", "tokenCount": 18},
    {"kind": "validated_fact", "content": "系统=UOS 20; 设备型号=...", "tokenCount": 24},
    {"kind": "assistant_action", "content": "已建议检查架构匹配与旧驱动残留", "tokenCount": 31}
  ],
  "slots": {"os": "UOS 20", "deviceModel": "..."},
  "updatedAt": "2026-08-27T01:00:00Z"
}
```

- 上下文条目不是界面聊天消息的镜像：可以只保留意图、已确认事实、决策、有效工具结果和回答要点，省略礼貌用语、重复内容和无效 delta。
- 只有完成且通过 output_guard 的结果才能沉淀为 assistant_action；FAILED/INTERRUPTED 的不完整文本不能进入后续上下文。
- 上下文预算建议：系统提示 15%、摘要 15%、最近对话 25%、检索/工具证据 30%、输出预留 15%；按模型窗口动态换算。
- 超过预算时先淘汰低价值 context entry，再更新摘要；摘要必须带 summaryThroughContextSeq，避免重复总结。
- 更新采用 session lock + contextVersion CAS，防止同一会话并发请求覆盖上下文。

7.3 上下文压缩与总结

上下文压缩由 LangGraph 中独立的 context_compaction 节点完成，不与最终回答生成混在同一次自由输出中。该节点读取结构化 context entries，输出可校验的摘要、事实、未解决问题和淘汰边界。

触发条件	处理
token 预算达到上下文窗口的 60%	在本轮检索前压缩最旧一段 context entries，保证证据与输出预算。
未压缩条目 >= 20 或新增 >= 8 轮	后台生成增量摘要，保留最近 6~12 轮高价值条目。
话题明显切换	封存上一主题摘要，新建 topic segment，避免旧主题干扰。
会话空闲 >= 30 分钟	异步压缩并把 Redis 热上下文写回 MongoDB，随后允许 TTL 过期。
工具结果过大	立即提取结论、关键字段、来源和有效期；原始结果只在受限 run 记录中短期保留。

- 摘要输出固定结构：goal、confirmed_facts、decisions、actions_taken、open_questions、constraints、source_context_seq_range。
- 摘要不得新增原上下文中不存在的事实；事实项保留 provenance/contextSeq 和置信度，低置信内容进入 open_questions。
- 压缩采用 append-new + CAS 切换 activeSummaryVersion，成功后再淘汰 Redis 旧条目；失败时保留旧摘要和原条目。
- 摘要质量通过离线“信息保持率、事实矛盾率、后续任务成功率”评测；不能只以 token 压缩率判断。

7.4 MongoDB 持久化模型

集合	主键/索引	用途
agent_context_entries	{sessionId, contextSeq} unique	规范化意图、已确认事实、决策、回答要点、检索/工具观察；不等同展示消息。
agent_runs	_id=requestId; {sessionId, createdAt}	运行状态、路由、prompt/model/config 版本、usage、终止原因。
agent_checkpoints	{requestId, checkpointNo} unique; TTL 可选	节点状态、恢复点、待确认工具调用；建议保留 7~30 天。
conversation_summaries	{sessionId, version} unique	滚动摘要、覆盖 contextSeq 范围、摘要模型、事实清单与质量标记。
user_memories	{userId, namespace, key} unique	经策略允许的跨会话偏好；默认关闭自动写入。

LangGraph 检查点建议直接采用 MongoDBSaver (langgraph-checkpoint-mongodb) 并以 sessionId 作为 thread_id、requestId 作为本轮 run 关联字段；业务化 context entries 和 conversation summaries 仍使用独立集合，避免把框架内部 checkpoint 结构直接当作稳定业务模型。依赖版本应在锁文件中固定并通过恢复测试后升级。

```
from langgraph.checkpoint.mongodb import MongoDBSaver

checkpointer = MongoDBSaver(
    mongo_client,
    db_name="xc_agent_context",
    checkpoint_collection_name="agent_checkpoints",
    writes_collection_name="agent_checkpoint_writes",
)
customer_service_graph = graph.compile(checkpointer=checkpointer)
```

7.5 与 MySQL chat_message 的严格边界

确定性边界 MySQL chat_message 永远只保存用户可查阅的提问和最终展示答案；Agent 组装上下文时不得读取该表。Redis/MongoDB 维护独立的上下文表示，允许筛选、改写、结构化和压缩，因此不要求与界面消息逐条对应。

1. 请求初始化先在 MySQL 保存本次用户提问与 request，不创建助手占位；随后把规范化后的 user_intent/context entry 幂等写 MongoDB，MongoDB 失败则不进入模型执行。
2. 生成完成后，Python 先持久化 Agent context、摘要、工具观察、checkpoint/run，并刷新 Redis；Java 再把最终展示答案、引用和 request 终态写 MySQL。
3. Redis 更新失败不影响 MongoDB 中的持久上下文；下一轮只从 MongoDB 重建。MySQL 不作为上下文回源。
4. 两套数据通过 requestId/sessionId 关联但不做逐字段对账；只对本轮是否完成、上下文版本是否提交、最终展示答案是否持久化做状态一致性检查。

7.6 隐私与保留

- 对手机号、身份证、序列号、IP 等字段分类；日志与 Trace 默认只保留摘要/哈希，正文单独授权。
- MongoDB 启用传输加密、静态加密、最小权限账号；上下文中不必要的敏感信息应在写入前移除，必要字段使用应用层或字段级加密。
- 会话软删除后立即从 Redis 清理，并在 MongoDB 标记 deletedAt；按保留策略异步物理清理，审计数据单独保留。
- 长期记忆不得从普通对话自动沉淀敏感事实；写入前需要策略判断，必要时征得用户同意，并提供删除能力。

8. 公共聊天接口详细设计

基地址 /api/v1。除登录接口外均要求 Authorization: Bearer <accessToken>。普通成功响应为 {requestId, data}；错误响应为 application/problem+json。资源归属一律取 JWT 上下文，不接收客户端 userId。

8.1 查询会话列表

项目	说明
方法与路径	GET /api/v1/sessions
当前状态	已实现
认证/幂等	Bearer；只读，无需幂等键

接收参数

参数	位置/类型	约束与含义
page	query / integer	可空，默认 1，最小 1
pageSize	query / integer	可空，默认 20，1~100
keyword	query / string	可空，trim 后最大 50；匹配 title/preview

实现思路

- 1. 从 AuthContext 取得 userId。
- 2. 按 user_id + deleted_at IS NULL 查询；updated_at DESC、id DESC 保证稳定排序。
- 3. COUNT 使用同一过滤条件，不能以当前页长度代替 total。

返回值

- 200：data={items:[{id,title,preview,updatedAt}],total,page,pageSize}。

失败情况

- 400 VALIDATION_FAILED；401 认证错误；503 DEPENDENCY_UNAVAILABLE。

8.2 修改会话标题

项目	说明
方法与路径	PATCH /api/v1/sessions/{sessionId}
当前状态	已实现；幂等头处理待补
认证/幂等	Bearer；建议要求 Idempotency-Key

接收参数

参数	位置/类型	约束与含义
sessionId	path / int64	必填, >0, 对应 chat_session.id
title	body / string	必填, trim 后 1~30 字符
Idempotency-Key	header / string	建议必填, 最大 128

实现思路

- 1. 校验 sessionId 与 title。
- 2. 以 sessionId + JWT userId + deleted_at IS NULL 更新, version+1。
- 3. 重新查询数据库最终值; 接入 api_idempotency_record 后缓存同键响应。

返回值

- 200: 更新后的 ChatSession。

失败情况

- 400 VALIDATION_FAILED; 404 CHAT_SESSION_NOT_FOUND; 409 IDEMPOTENCY_KEY_CONFLICT。

8.3 删除会话

项目	说明
方法与路径	DELETE /api/v1/sessions/{sessionId}
当前状态	已实现; Redis/Mongo 清理待补
认证/幂等	Bearer; 建议要求 Idempotency-Key

接收参数

参数	位置/类型	约束与含义
sessionId	path / int64	必填, >0
Idempotency-Key	header / string	建议必填

实现思路

- 1. 按 sessionId + userId 软删除 MySQL 会话。

- 2. 提交后删除 Redis 热上下文、SSE 重放和锁 Key。
- 3. MongoDB 对应 Agent context、summary、run/checkpoint 写 deletedAt；物理清理由保留策略任务执行。

返回值

- 204 No Content。重复删除可返回 204，或在统一策略下返回 404；推荐幂等 204。

失败情况

- 404 CHAT_SESSION_NOT_FOUND；409 状态冲突；503 存储依赖不可用。

8.4 查询历史消息

项目	说明
方法与路径	GET /api/v1/sessions/{sessionId}/messages
当前状态	部分实现；引用批量查询待补
认证/幂等	Bearer；只读

接收参数

参数	位置/类型	约束与含义
sessionId	path / int64	必填，>0
page	query / integer	默认 1，最小 1
pageSize	query / integer	默认 50，1~100

实现思路

- 1. 先用 sessionId + userId 校验归属，未命中统一返回 404。
- 2. 只从 MySQL 查询用户可见消息页，再以 messageIds 批量查询 citation 和当前用户 feedback，避免 N+1。
- 3. 该接口不读取 Redis/MongoDB Agent 上下文；用户界面记录和模型上下文保持解耦。

返回值

- 200：data={items:[ChatMessage],total,page,pagesize}; ChatMessage 包含 id,requestId,role,content,status,createdAt,stage,citations,feedback。

失败情况

- 400 VALIDATION_FAILED; 404 CHAT_SESSION_NOT_FOUND; 503 DEPENDENCY_UNAVAILABLE。

8.5 发起流式聊天

项目	说明
方法与路径	POST /api/v1/chat/stream
当前状态	契约与前端已存在，Java 后端待实现
认证/幂等	Bearer; Idempotency-Key 必填; Accept: text/event-stream

接收参数

参数	位置/类型	约束与含义
sessionId	body / int64 null	为空创建新会话; 非空必须属于当前用户
message	body / string	必填, 1~8000 字符
Idempotency-Key	header / string	必填, 1~128; 同用户同作用域唯一

实现思路

1. 在入口完成 JWT、参数、频率、并发配额和幂等校验; 同键同请求返回已有 request 状态, 同键异请求返回 409。
2. MySQL 短事务创建/校验 session、chat_request(ACCEPTED) 和用户提问消息(COMPLETED); 不创建助手占位消息, 提交后立即发送 meta。
3. 将不可变 ChatExecutionContext 交给专用有界线程池; 通过 HMAC 调用 Python /internal/ai/v1/chat/stream。
4. 逐事件校验 requestId、sequence、事件白名单和 payload 后转发; delta 只写缓冲, 不逐 token 落库。
5. 成功时 Python 先持久化 MongoDB Agent context/run 并刷新 Redis; Java 再在 MySQL 短事务写最终展示答案、引用、request/session, 最后发 citation、usage、done。

返回值

- 200 text/event-stream; 首个业务事件必须为 meta, 终态必须且只能为 done 或 error。
- 新会话时 meta.sessionId 返回服务端创建的主键; assistantMessageId 在 meta 中应改为可空/移除, 并只在 done 中返回最终消息 ID。

失败情况

- 建立 SSE 前可返回 400/401/404/409/429/503 Problem；建立后通过 error 事件返回 MODEL_UNAVAILABLE、KNOWLEDGE_UNAVAILABLE、STREAM_INTERRUPTED 等。

示例

```
POST /api/v1/chat/stream
Idempotency-Key: 3d6d9c21-...
Content-Type: application/json
Accept: text/event-stream

{"sessionId": null, "message": "UOS 升级后打印机无法识别怎么办?"}
```

8.6 查询聊天请求结果

项目	说明
方法与路径	GET /api/v1/chat/requests/{requestId}
当前状态	待实现
认证/幂等	Bearer；只读

接收参数

参数	位置/类型	约束与含义
requestId	path / int64	必填，>0；对应 chat_request.id

实现思路

- 按 requestId + JWT userId 查询，禁止枚举他人请求。
- 只从 MySQL 组装 request status、sessionId、assistantMessageId、最终展示 answer、citations、error、startedAt、finishedAt。
- MongoDB Agent 上下文不可用不应改变已经成功持久化的用户可见结果；但必须告警并阻止下一轮错误续聊。

返回值

- 200：ChatRequestResultVO；RUNNING/ACCEPTED 时 answer 可为空，SUCCEEDED 时 answer/citations 可用。

失败情况

- 404 CHAT_REQUEST_NOT_FOUND； 503 DEPENDENCY_UNAVAILABLE。

8.7 取消聊天请求

项目	说明
方法与路径	POST /api/v1/chat/requests/{requestId}/cancel
当前状态	待实现； 前端已调用
认证/幂等	Bearer； Idempotency-Key 必填

接收参数

参数	位置/类型	约束与含义
requestId	path / int64	必填, >0
Idempotency-Key	header / string	必填

实现思路

1. 按 requestId + userId 校验归属并读取当前状态。
2. ACCEPTED/RUNNING 写 Redis xc:chat:cancel:v1:{requestId}=1, TTL 5 分钟, 并调用 Python 内部取消。
3. 用状态条件更新 MySQL: ACCEPTED/RUNNING → CANCELLED; 由于没有助手占位消息, 无需写入失败/中断回答。
4. 已终态请求按幂等语义返回当前状态, 不覆盖 SUCCEEDED/FAILED。

返回值

- 200: 当前 ChatRequestResultVO; 也可统一为 204, 但需与 OpenAPI/前端固定一种, 建议保留当前 OpenAPI 的 200。

失败情况

- 404 CHAT_REQUEST_NOT_FOUND; 409 CHAT_REQUEST_STATE_CONFLICT; 503 AGENT_SERVICE_UNAVAILABLE。

8.8 设置或取消消息反馈

项目	说明
方法与路径	PUT /api/v1/messages/{messageId}/feedback
当前状态	待实现； DTO/表/前端已存在
认证/幂等	Bearer； Idempotency-Key 必填

接收参数

参数	位置/类型	约束与含义
messageId	path / int64	必填， >0
feedback	body / enum null	up、down 或 null； null 表示取消
Idempotency-Key	header / string	必填

实现思路

- 1. 通过 message → session 校验消息属于当前用户， 且 role=assistant、status=COMPLETED。
- 2. 以 (message_id,user_id) 唯一键 upsert； feedback=null 可删除行或写 NULL， 推荐删除以简化统计。
- 3. 写入反馈事件供离线评测按模型/prompt/知识版本聚合。

返回值

- 204 No Content， 与当前 OpenAPI 和前端一致。

失败情况

- 404 CHAT_MESSAGE_NOT_FOUND； 409 CHAT_MESSAGE_NOT_FEEDBACKABLE； 400 VALIDATION_FAILED。

8.9 SSE 事件契约

event	payload	约束
meta	sessionId,userMessageId	第一条； requestId 在外层； 不创建助手占位消息
status	stage,message	stage=queued/safety/intent/retrieval/generation/validation
delta	content	非空； 只包含新增片段
citation	sources[]	title,sourceId?,snippet,sourceLocator,page?

event	payload	约束
usage	model,promptTokens,completionTokens,totalTokens,estimate dCost	可选， 但生产建议发送
heartbeat	serverTime?	约 15 秒无业务事件时发送； 不持久化为消息
done	finishReason,messageId	唯一成功终态； 发送后关闭
error	code,message,retryable	唯一失败终态； 不得泄露内部异常

```
event: delta
data: {"event":"delta","requestId":21,"sequence":6,
data:  "occurredAt":"2026-08-27T01:10:03Z","payload":{"content":"请先确认系统版本。"}}

```

9. Java 与 Python 内部接口

内部基地址 /internal/ai/v1， 仅允许专用网络访问。所有请求使用 TLS + HMAC， 浏览器不得直接访问， 也不得传入 modelRoute、toolsEnabled、knowledgeBaseIds 等策略字段。

9.1 Agent 流式执行

项目	说明
方法与路径	POST /internal/ai/v1/chat/stream
当前状态	Java DTO 已定义； Controller/Service/Python 待实现
认证/幂等	TLS + HMAC； X-Internal-Key-Id/Timestamp/Nonce/Signature

接收参数

参数	位置/类型	约束与含义
requestId	body / int64	必填， >0
sessionId	body / int64	必填， >0
userId	body / int64	必填， 仅来自 Java 认证上下文
message	body / string	必填， 最大 8000
history	body / array	现有 DTO 兼容字段； Java 固定传空， 后续版本删除， 禁止从 MySQL 填充
policy	body / object	modelRoute、非空 knowledgeBaseIds、toolsEnabled、maxOutputTokens>=1

实现思路

- 1. Python 验证 HMAC 时间窗和 Redis nonce，校验 requestId 未被取消。
- 2. 按 sessionId 从 Redis 加载热上下文；未命中只从 MongoDB 回源，创建/恢复 LangGraph run/checkpoint。
- 3. 执行第 4 章状态图，输出与公共 SSE 同构的事件；Java 仍负责最终转发和 MySQL 用户可见记录持久化。

返回值

- 200 text/event-stream；事件结构与公共 SSE 一致。

失败情况

- 401 INTERNAL_SIGNATURE_INVALID；409 RUN_ALREADY_TERMINAL；429 AGENT_CAPACITY_EXCEEDED；503 MODEL/KNOWLEDGE_UNAVAILABLE。

9.2 取消 Agent 运行

项目	说明
方法与路径	POST /internal/ai/v1/chat/requests/{requestId}/cancel
当前状态	待实现
认证/幂等	TLS + HMAC；幂等

接收参数

参数	位置/类型	约束与含义
requestId	path / int64	必填
reason	body / string	可选：USER_CANCELLED/CLIENT_GONE/TIMEOUT

实现思路

- 1. 写共享取消令牌并通知本进程任务。
- 2. 在模型流、检索和每次工具调用边界检查取消。
- 3. 保存 checkpoint/run 终态；不把未完成 delta 作为后续上下文。

返回值

- 200/204：已取消或此前已终止。

失败情况

- 404 RUN_NOT_FOUND 可按幂等策略转 204；401 签名失败。

9.3 接收知识索引任务

项目	说明
方法与路径	POST /internal/ai/v1/index-jobs
当前状态	DTO 已定义；实现待补
认证/幂等	TLS + HMAC； jobId + attemptNo 幂等

接收参数

参数	位置/类型	约束与含义
jobId	body / string	必填
attemptNo	body / int	>=1
knowledgeBaseId	body / string	必填
documentId/versionId	body / string	必填
objectKey/sha256/fileName	body / string	必填
indexConfigVersion	body / string	必填

实现思路

1. 校验同一 job/attempt 是否已处理。
2. 异步执行解析、切片、Embedding、Qdrant upsert 和质量检查。
3. 通过回调报告结果；旧 attempt 不得覆盖新版本。

返回值

- 202：{jobId,status:'ACCEPTED'}。

失败情况

- 400 INDEX_JOB_INVALID; 409 INDEX_ATTEMPT_STALE; 503 INDEX_QUEUE_UNAVAILABLE。

9.4 管理端试问评测

项目	说明
方法与路径	POST /internal/ai/v1/evaluations/try
当前状态	DTO/VO 已定义；实现待补
认证/幂等	TLS + HMAC；仅 Java 管理端可发起

接收参数

参数	位置/类型	约束与含义
question	body / string	必填
modelRoute	body / string	可选
knowledgeBaseIds	body / array	可选，由 Java 管理权限限制
toolsEnabled	body / boolean	可选，默认 false
overrides	body / object	仅允许白名单评测参数

实现思路

1. 使用隔离的 evaluation run，不写入用户会话。
2. 返回 answer、实际 modelRoute、召回 chunks、refusalReason、configVersion 和 usage。
3. 评测调用默认禁用写工具。

返回值

- 200: EvaluationResultVO。

失败情况

- 400 EVALUATION_INVALID; 403 OVERRIDE_NOT_ALLOWED; 503 MODEL_UNAVAILABLE。

9.5 Python → Java 回调

接口	接收参数	实现与返回
POST /internal/business/v1/index-jobs/{jobId}/result	IndexJobResultDTO: jobId,attemptNo,status,documentVersionId,collection,chunkCount,embeddingModel,indexConfigVersion,errorCode,errorMessage	按 jobId+attemptNo 幂等; 仅当前 attempt 可更新; 200/204 确认。
POST /internal/business/v1/usage/batches	UsageBatchDTO: batchId, items[{callId,requestId,model,kind,tokens,cost,latencyMs,status,occurredAt}]	按 batchId/callId 去重; 返回 acceptedCount、rejectedCount、rejectedCallIds。

9.6 HMAC 规范

```
canonical = method + "\n" + canonicalPathAndQuery + "\n"
            + timestamp + "\n" + nonce + "\n" + sha256(body)
signature = hex(HMAC-SHA256(secret, canonical))
```

- Query 名称和值排序并统一百分号编码; body 使用原始字节计算 SHA-256。
- 允许时钟偏差不超过 300 秒; nonce 在 Redis 保存 300 秒, 重复立即拒绝。
- 支持 keyId 双密钥轮换; 比较使用恒定时间; 外部 Nginx 清除所有 X-Internal-* 头。

10. SSE、状态机、一致性与异常处理

10.1 请求状态与可见消息写入

```
chat_request:
ACCEPTED -> RUNNING -> SUCCEEDED
                -> FAILED
                -> CANCELLED
                -> INTERRUPTED

chat_message:
用户提问在请求初始化时写入 COMPLETED;
助手回答只在成功完成后写入 COMPLETED; 失败/取消不创建助手消息。
```

所有状态更新 SQL 必须带前置状态条件, 例如 UPDATE ... WHERE id=? AND status IN ('ACCEPTED','RUNNING')。这样可避免取消与成功并发提交时互相覆盖。终态不可逆; 需要重试时创建新 requestId, 并用 parentRequestId (建议新增) 关联。

10.2 事务边界

1. 初始化事务: 会话、request、用户提问消息、幂等占位一次提交; 不创建助手占位, 不得持有事务等待模型。

- 2. 运行阶段：delta 仅在内存/可选 Redis Stream 做短期重放，不逐 token 写 MySQL；MongoDB 只在节点检查点或上下文提交时写入。
- 3. 成功阶段：Python 幂等提交 MongoDB Agent context/run 并刷新 Redis；Java 写 MySQL 最终展示答案、citations、request、session preview；最后发送 done。
- 4. 失败/取消事务：只写 request 终态和安全错误摘要，不写失败/中断助手消息；已产生的 delta 可用于受限故障审计，但不进入对话上下文。

10.3 SSE 断线恢复

- 浏览器断线后可先 GET /chat/requests/{requestId}；SUCCEEDED 直接显示最终消息，RUNNING 可轮询或重新订阅。
- 可选 V1.1：Redis Stream 保存最近 10 分钟事件，Key=xc:chat:sse:v1:{requestId}，MAXLEN 500；客户端携带 Last-Event-ID 续传。
- 首个 delta 之后模型失败不得自动从头切换模型并重复输出；应终止为 error，或在服务端完整缓冲模式下重新生成。
- 网络断开默认允许后台继续生成；用户明确点击停止才触发 cancel。

10.4 错误映射

HTTP/场景	错误码	客户端行为
400	VALIDATION_FAILED	定位字段并提示修改
401	AUTH_TOKEN_MISSING/INVALID/EXPIRED	执行单飞 refresh；失败跳登录
404	CHAT_SESSION/REQUEST/MESSAGE_NOT_FOUND	不区分不存在和无权限
409	IDEMPOTENCY_KEY_CONFLICT / CHAT_REQUEST_STATE_CONFLICT	不自动重复写请求
429	CHAT_CAPACITY_EXCEEDED	退避后重试或稍后提交
503	MODEL_UNAVAILABLE / KNOWLEDGE_UNAVAILABLE / DEPENDENCY_UNAVAILABLE	显示可重试提示或工单入口
SSE error	STREAM_INTERRUPTED / TOOL_* / OUTPUT_VALIDATION_FAILED	停止 loading，保留 requestId 供恢复

11. 代码结构与实施步骤

11.1 Java 建议新增/调整

xc_agent/src/main/java/com/xc/agent/
controller/ChatController.java

补齐 stream/request/cancel/feedback

controller/InternalBusinessController.java
service/chat/ChatCommandService.java
service/chat/ChatStreamGateway.java
service/chat/ChatCancellationService.java
service/agent/AgentContextCoordinator.java
service/agent/AgentClient.java
repository/chat/ChatMessageRepository.java
mapper/ChatRequestMapper.java
mapper/MessageCitationMapper.java
mapper/MessageFeedbackMapper.java
config/ChatProperties.java
config/ChatExecutorConfig.java

索引结果、usage 回调
初始化/终态短事务
SSE 连接、事件校验与转发
Redis 取消令牌
协调 Python 上下文提交状态
HMAC WebClient/RestClient
仅 MySQL 用户可见消息
补齐状态机 SQL
batch insert/select
upsert/delete
超时、并发、TTL、预算
专用有界线程池

11.2 Python Agent 服务建议

agent_service/
app/main.py
api/internal_chat.py
graph/customer_service.py
graph/nodes/{guard,route,retrieve,tools,answer,validate}.py
models/state.py
chains/{routing,query_rewrite,answer}.py
memory/{redis_hot,mongo_store,context_builder}.py
retrieval/{rewrite,hybrid,rerank,evidence}.py
tools/{registry,gateway,schemas}.py
providers/{chat,embedding,reranker}.py
security/{hmac,pii,prompt_injection}.py
observability/{tracing,metrics}.py
tests/{unit,contract,integration,evals}/

LangChain + LangGraph
FastAPI 启动与生命周期
内部 SSE / cancel
LangGraph StateGraph 定义
AgentState/Pydantic schemas
LangChain LCEL/结构化输出

11.3 分阶段实施计划

阶段	交付	完成标准
P0 契约冻结	修正反馈枚举、幂等头、SSE schema、错误码	OpenAPI/JSON Schema 契约测试通过
P1 Java 写链路	4 个缺失公共接口、Mapper、状态机、引用/反馈	Mock AI 下端到端聊天成功
P2 上下文存储	Redis 热上下文、Mongo Agent context/summary/checkpoint、压缩策略	Redis 丢失只从 Mongo 重建，MySQL 不参与上下文
P3 Agent 骨架	Python 服务、HMAC、LangChain、LangGraph StateGraph、FAQ 快路径、流式事件	固定问答集与取消/超时/恢复通过
P4 RAG	索引、Qdrant hybrid、RRF、重排、引用门控	离线 Recall/Precision/引用正确率达标
P5 工具	只读工具、工单草稿/确认提交、审计	越权/注入/重复提交测试通过
P6 生产化	追踪、指标、预算、降级、容量、灾备	压测、故障注入和恢复演练通过

11.4 关键配置建议

app:
 chat:
 stream-timeout: 120s
 heartbeat-interval: 15s
 max-message-chars: 8000
 max-tool-calls: 3

```
tool-total-timeout: 10s
hot-context-ttl: 2h
cancel-ttl: 5m
sse-replay-ttl: 10m
context-recent-turns: 12
async:
  core-pool-size: 4
  max-pool-size: 8
  queue-capacity: 40
```

12. 测试、评测、可观测与验收

12.1 自动化测试

层级	重点
单元测试	状态迁移、上下文裁剪、路由结构化输出、RRF、证据门控、工具风险策略、SSE sequence。
契约测试	OpenAPI、Internal DTO、SSE JSON Schema；首事件 meta、sequence 递增、唯一 done/error。
集成测试	Testcontainers MySQL/Redis/MongoDB/Qdrant；跨库失败、幂等重放、Redis miss 回源。
端到端	新会话、连续问答、刷新恢复、取消、反馈、引用跳转、工单确认提交。
安全测试	跨用户 ID、提示词注入、工具参数越权、SSRF/SQL 注入、HMAC 重放、日志泄密。
故障注入	模型 429/5xx、Qdrant/Redis/Mongo/MySQL 不可用、SSE 断线、Python 重启。

12.2 AI 离线评测

指标	建议门槛/判定
意图准确率	>=95%；高风险写意图召回率优先
Retrieval Recall@20	>=90%，按产品线/问题类型分桶
Rerank nDCG@5	相对无重排基线显著提升
引用正确率	>=95%；引用必须支持对应断言
有依据回答率	>=95%；证据不足时正确澄清/拒答
工具成功率	>=99%（排除外部依赖故障）；零越权写入
用户反馈	赞率、重复提问率、转工单率按模型/prompt/知识版本跟踪

12.3 可观测

- Trace: requestId 为端到端 trace 关联键; 节点、模型、检索、工具、持久化分别建 span。正文和敏感工具结果默认不进入 Trace。
- Metrics: 请求量、TTFT、总耗时、各节点耗时、SSE 活跃数、队列长度、取消率、检索命中率、工具错误率、token/成本。
- Logs: 结构化记录
requestId/sessionId/userHash/modelRoute/promptVersion/indexConfigVersion/finalStatus; 禁止记录 JWT、Cookie、密钥和完整正文。
- Audit: 写工具的发起者、确认内容、工具名、参数摘要、资源 ID、结果和幂等键必须可追溯。

12.4 上线验收清单

- 8 个公共聊天接口与 6 个内部/回调接口均有契约测试; 实现状态不再存在文档与代码偏差。
- 同一 Idempotency-Key 并发提交只产生一组 session/request/message; 不同正文复用同键返回 409。
- SSE 任意路径都只有一个终态事件; 心跳不写入消息; 取消后不再输出 delta。
- Redis 全量清空后只从 MongoDB 恢复 Agent 上下文; MySQL 聊天记录不参与; MongoDB 短时故障不会产生假成功。
- 知识回答引用可定位到已发布文档版本和页码; 旧版本 chunk 不进入回答。
- 提示词注入不能改变系统指令、扩展知识库范围或触发未授权工具。
- 并发和超时达到项目容量目标, 队列满时返回 429 而不是无限堆积。

附录 A. 数据结构建议

A.1 MongoDB agent_context_entries 示例

```
{
  "_id": "ctx_3_27",
  "sessionId": 3,
  "requestId": 21,
  "contextSeq": 27,
  "kind": "assistant_action",
  "content": "已确认 UOS 20 与设备型号; 建议先清理旧驱动并安装对应 CPU 架构版本。",
  "contentHash": "sha256:...",
  "source": {"node": "output_guard", "displayMessageId": 102},
  "model": {"route": "customer-service-default", "name": "..."},
  "facts": [{"key": "os", "value": "UOS 20", "confidence": 1.0}],
  "tokenCount": 31,
  "createdAt": "2026-08-27T01:10:00Z",
  "schemaVersion": 1
}
```

A.2 MongoDB agent_runs 示例

```
{
  "_id": 21,
```

```
"sessionId": 3,
"userIdHash": "...",
"status": "SUCCEEDED",
"currentNode": "finalize",
"intent": {"type": "DIAGNOSIS", "confidence": 0.94},
"modelRoute": "customer-service-default",
"promptVersion": "support-v3",
"indexConfigVersion": "kb-index-v2",
"toolCalls": [{"callId": "...", "tool": "device_compatibility_lookup", "status": "OK"}],
"usage": {"promptTokens": 286, "completionTokens": 96, "totalTokens": 382},
"startedAt": "...",
"finishedAt": "...",
"schemaVersion": 1
}
```

A.3 建议新增的 MySQL 字段/表

对象	建议
chat_request	增加 parent_request_id、agent_run_version、prompt_version、index_config_version、last_sequence。
chat_message	继续保存用户提问与最终展示答案；可增加 answer_version/context_commit_version 仅用于完成状态关联，不保存 Agent 中间上下文。
chat_context_commit	request_id、session_id、context_version、mongo_run_id、status、committed_at；仅记录上下文提交结果，不复制上下文正文。
tool_execution	call_id、request_id、tool_name、risk_level、args_hash、status、latency_ms、resource_id、created_at。

附录 B. Redis Key 建议

Key 模式	TTL	用途
xc:chat:ctx:v1:{sessionId}	2h sliding	近期消息、摘要、槽位、contextVersion
xc:chat:lock:v1:{sessionId}	30s	同会话上下文更新短锁
xc:chat:cancel:v1:{requestId}	5m	取消令牌
xc:chat:sse:v1:{requestId}	10m	可选 Redis Stream 事件重放
xc:chat:idem:v1:{userId}:{key}	24h	聊天提交幂等快速层；MySQL 仍保存可靠记录
xc:chat:quota:v1:{userId}	窗口期	用户速率/并发计数
xc:internal:nonce:v1:{keyId}:{nonce}	5m	内部 HMAC 防重放
xc:faq:v1:answer:{questionHash}	10m	已发布 FAQ 快路径缓存

附录 C. 参考资料与项目依据

C.1 项目内依据

- `xc_agent/src/main/java/com/xc/agent/controller/ChatController.java`: 当前 4 个已实现公共接口。
- `xc_agent/src/main/java/com/xc/agent/service/impl/ChatServiceImpl.java`: 用户隔离、分页、软删除与消息转换。
- `xc_agent/src/main/java/com/xc/agent/model/dto/chat/ChatDTOs.java`: 聊天接收参数与校验。
- `xc_agent/src/main/java/com/xc/agent/model/dto/internal/InternalDTOs.java`: Java/Python 内部 DTO。
- `database/mysql/01_schema.sql`: 会话、请求、消息、引用、反馈和幂等表。
- 接口文档/`openapi.yaml`: 公共 API 契约。
- 接口文档/`sse-events.schema.json`: SSE 事件契约。
- `frontend/src/services/chat.ts`: 前端 SSE、恢复、取消与反馈调用。

C.2 外部官方参考

- [OpenAI Agents SDK: Agent orchestration](#): 代码编排与 LLM 编排、manager/agents-as-tools 与 handoff 的边界。
- [OpenAI Agents SDK: Tools](#): 函数工具、Agent-as-tool、工具超时与审批。
- [OpenAI Agents SDK: Tracing](#): generation、tool、guardrail、handoff 等端到端 trace/span。
- [LangGraph: Workflows and agents](#): 工作流与动态 Agent 的区别及常见模式。
- [LangGraph: Persistence](#): 检查点、线程状态、恢复、人工介入与容错。
- [LangChain: Memory overview](#): thread-scoped 短期记忆与跨会话长期记忆。
- [Qdrant: Hybrid and multi-stage queries](#): dense+sparse、RRF/DBSF 与多阶段查询。
- [MongoDB: Change Streams](#): 持久化变更订阅、恢复令牌及副本集约束。
- [MongoDB: Integrate MongoDB with LangGraph](#): MongoDBSaver 检查点、MongoDB Store 与 LangGraph 状态持久化。
- [Redis: Streams](#): 短期事件日志、消费与裁剪能力。
- [OWASP LLM06:2025 Excessive Agency](#): 工具最小权限、最小功能和最小自治。

说明: 本项目的编排框架确定为 LangChain + LangGraph; 其中 LangGraph 负责流程状态与恢复, LangChain 负责模型、Prompt、Retriever 和 Tool 适配。模型供应商、Embedding 和 reranker 仍通过 Provider Adapter 与评测数据选择。